

UNIVERSIDAD PRIVADA ANTENOR ORREGO
Facultad de Ingeniería de Sistemas e Inteligencia Artificial

Propuesta de Sistema de Búsqueda de Información en Documentos Universitarios usando la Técnica de IA RAG (Retrieval-Augmented Generation)

Trabajo individual — Investigación aplicada

Autor: Mostacero Cieza, Luis David

Docente: Mendoza Corpus, Carlos Alfredo

Fecha: 03 de julio de 2026

Repositorio con la solución: <https://github.com/FluidityCore/rag-matricula-universidad>

1. Introducción

La universidad requiere un programa que permita a los estudiantes buscar información dentro de un repositorio de documentos institucionales (reglamentos, requisitos, calendarios) formulando preguntas en lenguaje natural, por ejemplo: “*Requisitos para reserva de matrícula*”. Como propuesta de solución y línea base para un futuro producto de software institucional, se investigó y desarrolló un prototipo funcional basado en la técnica de inteligencia artificial conocida como **RAG (Retrieval-Augmented Generation)**.

Este informe documenta el marco teórico de RAG, la metodología y arquitectura del prototipo, los paquetes de software utilizados, los prompts empleados como apoyo de IA durante el desarrollo, los casos de prueba resueltos y las conclusiones, junto con el enlace al repositorio que contiene la solución completa en Python.

2. Marco teórico: ¿qué es RAG?

RAG (Retrieval-Augmented Generation, «generación aumentada por recuperación») es una técnica de IA que combina un sistema de recuperación de información con un modelo de lenguaje generativo (LLM), en lugar de depender únicamente del conocimiento memorizado por el modelo durante su entrenamiento.

Consta de dos etapas:

Retrieval (recuperación): el repositorio de documentos se divide en fragmentos (*chunks*), y cada fragmento se convierte en un vector numérico (*embedding*) que representa su significado. Estos vectores se almacenan en una base de datos vectorial. Ante una pregunta del usuario, esta también se convierte en un embedding y se buscan los fragmentos más parecidos por similitud semántica (no por coincidencia exacta de palabras).

Generation (generación): los fragmentos recuperados se entregan como contexto a un modelo de lenguaje, que redacta la respuesta final basándose en esa información. Esto reduce las «alucinaciones» (respuestas inventadas) y permite trazar la respuesta hasta su fuente documental.

RAG es especialmente adecuado para el caso de la universidad porque el reglamento y los procedimientos cambian con cada periodo académico: en lugar de re-entrenar un modelo, basta con actualizar los documentos del repositorio y reconstruir el índice vectorial.

3. Metodología y arquitectura del prototipo

Se optó por una implementación 100% local y gratuita (sin API de pago), adecuada para un prototipo académico reproducible. El flujo implementado es:

1. **Ingesta:** se leen los documentos .txt del repositorio (`repositorio_documentos/`).
2. **Chunking:** cada documento se agrupa por párrafos en fragmentos de hasta 600 caracteres, evitando cortar oraciones a la mitad.
3. **Embeddings:** cada fragmento se convierte en un vector con el modelo `paraphrase-multilingual-MiniLM-L12-v2`.

- 4. **Indexado:** los vectores se almacenan en un índice FAISS (IndexFlatIP) para búsqueda por similitud coseno.
- 5. **Recuperación:** ante una pregunta, se embebe y se buscan los 3 fragmentos más similares que superen un umbral mínimo de similitud.
- 6. **Generación:** los fragmentos recuperados se insertan en una plantilla de prompt y se envían al modelo local google/flan-t5-base, que redacta la respuesta final.

Estructura de archivos del proyecto: repositorio_documentos/ (fuentes), rag/ingesta.py, rag/recuperador.py, rag/generador.py, rag/config.py y main.py como interfaz de línea de comandos.

4. Paquetes y tecnologías utilizadas

Todos los paquetes son de código abierto y gratuitos. Las versiones exactas quedan fijadas en requirements.txt del repositorio.

Paquete	Version	Uso en el proyecto
sentence-transformers	5.6.0	Generacion de embeddings semanticos multilingues
faiss-cpu	1.14.3	Base de datos vectorial (busqueda por similitud)
transformers	5.13.0	Carga del modelo local de generacion de texto (LLM)
torch	2.12.1	Motor de computo para los modelos anteriores
numpy	2.5.0	Manejo de vectores y matrices de embeddings

Modelos de Hugging Face empleados (descarga automatica, sin costo): embeddings sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 y generacion google/flan-t5-base.

5. Prompts usados como apoyo de IA

El prototipo se diseño con el apoyo del asistente de IA Claude Code (Anthropic). A continuacion se listan los prompts principales usados durante el desarrollo (el detalle completo esta tambien en PROMPTS.md del repositorio).

Prompt 1 - Enunciado del trabajo

Enunciado original entregado por la universidad sobre el sistema de busqueda de informacion y la propuesta de RAG.

Prompt 2 - Alcance tecnico

“Ayudame a construir un prototipo en Python que implemente RAG para responder preguntas sobre un repositorio de documentos universitarios... 100% local y gratuito... con chunking, embeddings, indice vectorial, recuperacion y generacion... buenas practicas: type hints, pathlib.Path, funciones cortas de una sola responsabilidad.”

Prompt 3 - Seleccion de librerias

“¿Que combinacion de librerias Python recomiendas para implementar RAG de forma local y gratuita, sin depender de OpenAI ni Anthropic para los embeddings?”

Prompt 4 - Estructura del codigo

“Genera el codigo dividido en modulos: ingesta.py, recuperador.py, generador.py y main.py como CLI.”

Prompt 5 - Documentos de ejemplo

“No tengo acceso a los reglamentos reales de mi universidad. Genera 2 o 3 documentos de ejemplo realistas sobre reserva de matricula, requisitos y calendario academico.”

Prompt 6 - Validacion

“Ejecuta el flujo completo (indexar y preguntar) y muestrame los fragmentos recuperados y la respuesta generada.”

Prompt 6b - Correccion de caso limite

“Al preguntar algo fuera del repositorio, el sistema inventa una respuesta. Corrige el retriever para que aplique un umbral de similitud y, si nada lo supera, informe que no hay informacion suficiente.”

Prompt 7 - Redaccion del informe

“Con base en el codigo y los resultados, redacta el informe en PDF con introduccion, marco teorico, metodologia, paquetes, prompts, codigo, pruebas, conclusiones y el enlace al repositorio.”

6. Implementacion (evidencia en archivos .py)

El código completo está anexado en el repositorio de GitHub (sección 10) en los archivos `rag/ingesta.py`, `rag/recuperador.py`, `rag/generador.py`, `rag/config.py` y `main.py`. A continuación se muestran los dos fragmentos más representativos de la técnica RAG: la recuperación semántica con umbral de similitud, y la generación de la respuesta final.

`rag/recuperador.py` (recuperación semántica)

```
def recuperar_fragmentos(
    pregunta: str,
    modelo: SentenceTransformer,
    indice: faiss.Index,
    metadatos: list[dict[str, str]],
    top_k: int = TOP_K,
    umbral: float = UMBRAL_SIMILITUD,
) -> list[dict[str, str]]:
    """Embebe la pregunta y devuelve los top_k fragmentos más similares
    del índice, descartando los que queden por debajo del umbral."""
    embedding_pregunta = modelo.encode(
        [pregunta], convert_to_numpy=True, normalize_embeddings=True
    )
    similitudes, posiciones = indice.search(
        embedding_pregunta.astype(np.float32), top_k
    )
    return [
        metadatos[i]
        for i, similitud in zip(posiciones[0], similitudes[0])
        if i != -1 and similitud >= umbral
    ]
```

`rag/generador.py` (generación de la respuesta)

```
def generar_respuesta(
    pregunta: str, fragmentos: list[dict[str, str]], generador: ModeloGeneracion
) -> str:
    """Genera la respuesta final invocando el modelo local con el
    prompt construido a partir de los fragmentos recuperados."""
    prompt = construir_prompt(pregunta, fragmentos)
    entradas = generador.tokenizador(prompt, return_tensors="pt", truncation=True)
    salida = generador.modelo.generate(
        **entradas, max_new_tokens=200, min_new_tokens=20
    )
    return generador.tokenizador.decode(salida[0], skip_special_tokens=True).strip()
```

7. Casos de prueba y resultados

Caso 1 - Pregunta directa del enunciado. El retriever recupera exactamente el artículo del reglamento con los requisitos y el paso a paso del procedimiento; el modelo genera una respuesta que resume los requisitos citando ambas fuentes.

```
$ python main.py preguntar "Requisitos para reserva de matricula"
```

```
Fragmentos recuperados:
[reglamento_matricula.txt] Artículo 11.- Requisitos para reserva de
matricula: a) Ser estudiante regular con matricula vigente o haber
estado matri...
[requisitos_reserva_procedimiento.txt] Paso 4: Adjuntar los documentos
requeridos segun el motivo de la reserva: - Motivo laboral: carta de
la empresa o con...
[reglamento_matricula.txt] Artículo 12.- La solicitud de reserva de
```

matricula sera evaluada por la Oficina de Registros Academicos en un plazo maxi...

Respuesta generada por el modelo (se conservan sus imperfecciones de ortografia y gramatica, ver seccion 8 - Limitaciones):

Paso 4: Adjuntar documentos requeridos por reserva: a) ser estudiante regular con reserva vigente o adquirir estado matriculado en el ao anterior. b) Presentar la solicitud de reserva a traves del portal virtual de tramites, dentro de los plazos establecidos en el calendario academico. c) No registrar deudas en concepto de pensiones o derechos academicos al momento de la solicitud. d) Adjuntar el pago de la tasa administrativa por derecho de reserva.

Caso 2 - Pregunta sobre plazos. La pregunta “Hasta cuando puedo solicitar reserva de matricula” recupera el articulo sobre el plazo de resolucion (5 dias habiles) y el calendario academico con las fechas exactas de matricula e inicio de clases.

Caso 3 - Pregunta fuera del repositorio. Ante una pregunta sin relacion con los documentos (por ejemplo, sobre el clima), ningun fragmento supera el umbral de similitud configurado (`UMBRAL_SIMILITUD = 0.3`), por lo que el sistema informa que no encontro informacion suficiente en lugar de inventar una respuesta. Esta es una de las principales ventajas de RAG frente a un LLM sin contexto: reduce las alucinaciones.

```
$ python main.py preguntar "Cual es el clima en Lima manana"
```

No se encontro informacion suficiente en el repositorio de documentos.

8. Limitaciones y trabajo futuro

- El modelo local de generacion (`flan-t5-base`) tiene soporte multilingue limitado: produce respuestas comprensibles pero con errores menores de ortografia y gramatica en espanol. En un producto real se recomienda un modelo mas grande o una API de un LLM con mejor soporte de espanol.
- El repositorio de prueba usa documentos de ejemplo (no los reglamentos reales de la universidad); el siguiente paso es ingestar los PDF/Word oficiales.
- El umbral de similitud (0.3) se calibro de forma empirica; en produccion debe ajustarse con un conjunto real de preguntas de los estudiantes.
- Como producto futuro se propone exponer el prototipo como una API web, agregar autenticacion de usuarios, un panel para que la oficina de registros actualice el repositorio, y metricas de las preguntas mas frecuentes.

9. Conclusiones

Se disenio, implemento y probó un prototipo funcional de busqueda de informacion sobre documentos universitarios usando la tecnica RAG, cumpliendo el requerimiento de responder preguntas en lenguaje natural como “Requisitos para reserva de matricula” con informacion extraida directamente del repositorio de documentos.

La combinacion de recuperacion semantica (embeddings + FAISS) con un umbral de similitud y un modelo generativo local demuestra, con un stack 100% gratuito, los principios centrales de RAG: respuestas basadas en evidencia documental y mitigacion de alucinaciones ante preguntas fuera de alcance. Este prototipo constituye la base tecnica y conceptual para el desarrollo de un futuro producto de software institucional de atencion a consultas academicas.

10. Referencias y enlace de la solucion

- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Documentacion de sentence-transformers: <https://www.sbert.net>
- Documentacion de FAISS: <https://faiss.ai>
- Documentacion de Hugging Face Transformers: <https://huggingface.co/docs/transformers>
- Modelo flan-t5-base: <https://huggingface.co/google/flan-t5-base>

Repositorio con la solucion completa (codigo .py, documentos de ejemplo, README y PROMPTS.md):

<https://github.com/FluidityCore/rag-matricula-universidad>